LAPORAN PRAKTIKUM STRUKTUR DATA

Stack

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Stack. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 09 April 2026

Aliifah Felda Mufarrihati Salwaa
2511531011

DAFTAR ISI

KATA PENGANTAR ..... **i**
BAB I PENDAHULUAN **3**
1.1 Latar Belakang Praktikum 3
1.2 Tujuan Praktikum 3
1.3 Manfaat Praktikum 3
BAB II PEMBAHASAN **4**
2.1. Dasar Teori 4
2.2. Class stackArray_2511531011 5
2.3. stackArrayDriver_2511531011 6
2.4. stackPostfix_2511531011 7
2.5. NilaiMaksimum_2511531011 9
2.6. SiswaStack_2511531011 10
2.7. Tugas Pekan 3 12
BAB III KESIMPULAN **17**
3.1. Kesimpulan 17
3.2. Saran 17
DAFTAR PUSTAKA **18**

BAB I
PENDAHULUAN

1.1 Latar Belakang Praktikum

Setelah mempelajari salah satu implementasi ADT yakni array list pada praktikum sebelumnya, kita juga harus mengetahui tentang penerapan ADT yang lain. Karena banyaknya implementasi ADT, kita harus dapat menguasai setiap implementasinya agar dapat membangun sebuah program efektif dan efisien, namun tetap sesuai dengan keinginan kita.

Oleh karena itu, pada praktikum kali ini kita akan mempelajari dan memahami tentang penerapan ADT yang lainnya yakni Stack. Stack ini menggunakan prinsip LIFO (*Last In First Out*) dimana data yang terakhir masuk, itulah yang pertama keluar.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami tentang implementasi ADT yakni Stack.
2. Mengetahui kelebihan dan kekurangan Stack dalam program.
3. Dapat membangun sebuah kode program dengan Stack

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mampu mengimplementasikan Stack dalam pembuatan program agar lebih efektif dan efisien.
2. Memahami penggunaan Stack dalam program.

BAB II
PEMBAHASAN

2.1. Dasar Teori

Stack merupakan salah satu struktur data dalam Java yang digunakan untuk menyimpan sekumpulan elemen dengan prinsip LIFO (*Last In, First Out*), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Stack dapat diimplementasikan menggunakan kelas Stack yang tersedia dalam package java.util atau menggunakan struktur lain seperti array maupun LinkedList.

Operasi yang terdapat pada Stack adalah sebagai berikut:

1. Menambahkan elemen (Push): Kita dapat menggunakan push(object) untuk menambahkan elemen ke bagian atas stack. Elemen yang dimasukkan terakhir akan berada di posisi paling atas.
2. Menghapus elemen (Pop): Kita dapat menggunakan pop() untuk mengambil sekaligus menghapus elemen yang berada di posisi paling atas stack.
3. Melihat elemen teratas (Peek): Kita dapat menggunakan peek() untuk melihat elemen yang berada di posisi paling atas tanpa menghapusnya dari stack.
4. Mengecek kondisi stack: Kita dapat menggunakan isEmpty() untuk mengecek apakah stack kosong, serta size() untuk mengetahui jumlah elemen yang tersimpan dalam stack.

Stack memiliki beberapa kelebihan dibandingkan dengan struktur data lainnya. Stack memiliki implementasi yang sederhana dan efisien untuk kasus yang membutuhkan urutan terbalik (reverse order). Selain itu, operasi pada Stack seperti push dan pop memiliki kompleksitas waktu yang relatif cepat karena hanya berfokus pada satu sisi (top).

Namun, di balik kelebihannya, Stack juga memiliki kekurangan. Stack memiliki keterbatasan dalam akses data karena hanya elemen teratas yang dapat diakses

secara langsung. Selain itu, Stack tidak fleksibel untuk pencarian elemen di tengah data, serta dapat menyebabkan pemborosan memori jika tidak digunakan secara optimal.

2.2. Class stackArray_2511531011

```
package pekan3_2511531011;

public class stackArray_2511531011 {
	static final int MAX_1011 = 1000;
	int top_1011;
	int a_1011 [] = new int [MAX_1011];
	boolean isEmpty_1011() {
		return (top_1011 < 0);
	}

	stackArray_2511531011() {
		top_1011 = -1;
	}

	boolean push_1011 (int x_1011) {
		if (top_1011 >= (MAX_1011 - 1)) {
			System.out.println("Stack overflow.");
			return false;
		} else {
			a_1011 [++top_1011] = x_1011;
			System.out.println(x_1011 + " dimasukkan dalam
stack");
			return true;
		}
	}

	int pop_1011 () {
		if (top_1011 < 0) {
			System.out.println("Stack underflow");
			return 0;
		} else {
```

```
                int x_1011 = a_1011 [top_1011--];
                return x_1011;
            }
        }

        int peek_1011() {
            if (top_1011 < 0) {
                System.out.println("Stack underflow");
                return 0;
            } else {
                int x_1011 = a_1011 [top_1011];
                return x_1011;
            }
        }

        void print_1011() {
            for (int i_1011 = top_1011; i_1011 > -1; i_1011--) {
                System.out.print(" " + a_1011[i_1011]);
            }
        }
}
```

Kode program 2.1: stackArray_2511531011

Class ini merupakan ADT dan konstruktor yang akan digunakan pada class stackArrayDriver.

2.3. stackArrayDriver_2511531011

```
package pekan3_2511531011;

public class stackArrayDriver_2511531011 {

    public static void main(String[] args) {
        stackArray_2511531011 s_1011 = new stackArray_2511531011();
        s_1011.push_1011(10);
        s_1011.push_1011(20);
        s_1011.push_1011(30);
```

```
            System.out.println(s_1011.pop_1011() + " dikeluarkan dari
stack");
            System.out.println("Elemen teratas adalah : " +
s_1011.peek_1011());
            System.out.print("Elemen pada stack :");
            s_1011.print_1011();

      }

}
```

Kode program 2.2: stackArrayDriver_2511531011

Pada kode program ini, kita melakukan operasi terhadap elemen teratas dengan menggunakan stack, yakni operasi push untuk memasukkan angka, operasi pop untuk mengeluarkan angka, dan operasi peek untuk melihat angka paling atas.

Output :

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack : 20 10
```

2.4. stackPostfix_2511531011

```
package pekan3_2511531011;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511531011 {
      public static int postfixEvaluate_1011(String expression) {
            Stack <Integer> s_1011 = new Stack <Integer>();
            Scanner input_1011 = new Scanner(expression);
            while (input_1011.hasNext()) {
                  if (input_1011.hasNextInt()) { // an operand integer
                        s_1011.push(input_1011.nextInt());
```

```
                    } else { //an operator
                            String operator_1011 = input_1011.next();
                            int operand2_1011 = s_1011.pop();
                            int operand1_1011 = s_1011.pop();
                            if (operator_1011.equals("+")) {
                                    s_1011.push(operand1_1011 +
operand2_1011);
                            } else if (operator_1011.equals("-")){
                                    s_1011.push(operand1_1011 -
operand2_1011);
                            } else if (operator_1011.equals("*")){
                                    s_1011.push(operand1_1011 *
operand2_1011);
                            } else {
                                    s_1011.push(operand1_1011 /
operand2_1011);
                            }
                    }
            }
            input_1011.close();
            return s_1011.pop();
    }

    public static void main(String[] args) {
            System.out.println("hasil postfix = " +
postfixEvaluate_1011("5 2 4 * + 7 -"));
    }

}
```

Kode program 2.3: stackPostfix_2511531011

Pada kode program ini, kita akan mengimplementasikan stack untuk perhitungan aritmatika. Karena menggunakan stack, operand yang paling atas dioperasikan dengan operand yang berada dibawahnya.

Output :

```
hasil postfix = 6
```

2.5. NilaiMaksimum_2511531011

```java
package pekan3_2511531011;

import java.util.Stack;

public class NilaiMaksimum_2511531011 {
	public static int max_1011(Stack<Integer> s_1011) {
		Stack<Integer> backup_1011 = new Stack<Integer>();
		int maxValue_1011 = s_1011.pop();
		backup_1011.push(maxValue_1011);
		while (!s_1011.isEmpty()) {
			int next_1011 = s_1011.pop();
			backup_1011.push(next_1011);
			maxValue_1011 = Math.max(maxValue_1011, next_1011);
		}
		while (!backup_1011.isEmpty()) {
			s_1011.push(backup_1011.pop());
		}
		return maxValue_1011;
	}

	public static void main(String[] args) {
		Stack<Integer> s_1011 = new Stack<Integer>();
		s_1011.push(70);
		s_1011.push(12);
		s_1011.push(20);
		System.out.println("Isi stack "+s_1011);
		System.out.println("Stack teratas : "+s_1011.peek());
		System.out.println("Nilai maksimum : "+ max_1011(s_1011));

	}

}
```

Kode prorgam 2.4: NilaiMaksimum_2511531011

Kode program ini sejatinya merupakan implementasi stack paling sederhana, yakni memasukkan data dengan push, mencari nilai teratas dengan peek, dan mencari nilai max dengan method max.

Output :

```
Isi stack [70, 12, 20]
Stack teratas : 20
Nilai maksimum : 70
```

2.6. SiswaStack_2511531011

```java
package pekan3_2511531011;

import java.util.ArrayList;

class Siswa_1011 {
	String nama_1011;
	int nim_1011;

	public Siswa_1011(String nama_1011, int nim_1011) {
		this.nama_1011 = nama_1011;
		this.nim_1011 = nim_1011;
	}

	@Override
	public String toString() {
		return "Nim: " + nim_1011 + ", Nama: " + nama_1011;
	}
}

public class SiswaStack_2511531011 {
	private ArrayList<Siswa_1011> stack_1011;

	public SiswaStack_2511531011() {
		stack_1011 = new ArrayList<>();
	}
```

```java
	public void push_1011(Siswa_1011 mhs_1011) {
		stack_1011.add(mhs_1011);
	}

	public Siswa_1011 pop_1011() {
		if (!isEmpty_1011()) {
			return stack_1011.remove(stack_1011.size() - 1);
		} else {
			return null;
		}
	}

	public Siswa_1011 peek_1011() {
		if (!isEmpty_1011()) {
			return stack_1011.get(stack_1011.size() - 1);
		} else {
			return null;
		}
	}

	public boolean isEmpty_1011() {
		return stack_1011.isEmpty();
	}

	public void tampilkanSiswa_1011() {
		for (int i_1011 = stack_1011.size() - 1; i_1011 >= 0;
i_1011--) {
			System.out.println(stack_1011.get(i_1011));
		}
	}

	public static void main(String[] agrs) {
		SiswaStack_2511531011 studentStack_1011 = new
SiswaStack_2511531011();

		Siswa_1011 mhs1_1011 = new Siswa_1011("Ali", 1);
		Siswa_1011 mhs2_1011 = new Siswa_1011("Boby", 2);
```

```
            Siswa_1011 mhs3_1011 = new Siswa_1011("Charles", 3);

            studentStack_1011.push_1011(mhs1_1011);
            studentStack_1011.push_1011(mhs2_1011);
            studentStack_1011.push_1011(mhs3_1011);

            System.out.println("Siswa di dalam stack: ");
            studentStack_1011.tampilkanSiswa_1011();

            System.out.println("Siswa teratas " +
studentStack_1011.peek_1011());
            System.out.println("mengeluarkan siswa teratas dari stack:
" + studentStack_1011.pop_1011());
            System.out.println("Daftar siswa setelah di pop: ");
            studentStack_1011.tampilkanSiswa_1011();
      }

}
```

Kode program 2.5: SiswaStack_2511531011

Pada kode program ini kita akan memasukkan beberapa nama siswa ke dalam program menggunakan push, mencari nama siswa teratas dengan menggunakan peek, dan mengeluarkan nama siswa dengan menggunakan pop.

Output :

```
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
Daftar siswa setelah di pop:
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

2.7. Tugas Pekan 3

A. Class Website_2511531011

```java
package pekan3_2511531011;

public class Website_2511531011 {
	String judul_1011;
	String URL_1011;

	//Konstruktor
	Website_2511531011(String judul, String URL) {
		this.judul_1011 = judul;
		this.URL_1011 = URL;
	}

	@Override
	public String toString () {
		return "Judul: " + judul_1011 + ", URL: " + URL_1011;
	}

	//getter
	public String getJudul_1011() {
		return judul_1011;
	}

	public String getURL_1011() {
		return URL_1011;
	}


	//Setter
	public void setJudul_1011(String judul) {
		this.judul_1011 = judul;
	}

	public void setURL_1011(String URL) {
		this.URL_1011 = URL;
	}
}
```

Class ini digunakan untuk membuat ADT dan konstruktor yang akan digunakan pada class browser.

B. Class Browser_2511531011

```
package pekan3_2511531011;

import java.util.Stack;
import java.util.Scanner;

public class Browser_2511531011 {
	// 1. Method untuk pilih menu
		public static void tampilkanMenu_2511531011() {
			System.out.println("\n1. Kunjungi Website (Push)");
			System.out.println("2. Tombol Back (Pop)");
			System.out.println("3. Lihat Halaman Aktif (Peek)");
			System.out.println("4. Keluar");
		}

		// 2. Method untuk push
		public static void kunjungiWebsite_2511531011
(Stack<Website_2511531011> list, Scanner sc) {
			System.out.print("Masukkan Judul: ");
			String judul = sc.nextLine();
			System.out.print("Masukkan URL: ");
			String URL = sc.nextLine();
			Website_2511531011 web = new
Website_2511531011(judul, URL);
			list.push(web);
			System.out.println("Berhasil Mengunjungi Halaman!");
			}

		// 3. Method untuk pop
		public static void tombolBack_2511531011
(Stack<Website_2511531011> list) {
			if (list.isEmpty()) {
				System.out.println("Histori kosong");
			} else {
				Website_2511531011 web = list.pop();
```

```
                        System.out.println("Kembali dari: " +
web.getJudul_1011());
                    }
            }

            // 4. Method untuk peek
            public static void halamanAktif_2511531011
(Stack<Website_2511531011> list, Scanner sc) {
                    if (list.isEmpty()) {
                        System.out.println("Tidak ada halaman
aktif");
                    } else {
                        Website_2511531011 web = list.peek();
                        System.out.println("Halaman aktif: " +
web.getJudul_1011() + " " + web.getURL_1011());
                    }
            }

            public static void main (String[] args) {
                    Stack<Website_2511531011> browser = new Stack<>();
                    Scanner scanner = new Scanner(System.in);
                    int choice;

                    System.out.println("=== Browser History NIM:
2511531011 ===");

                    do {
                        tampilkanMenu_2511531011();
                        System.out.print("\nPrint Menu: ");
                        choice = scanner.nextInt();
                        scanner.nextLine();

                        switch (choice) {
                        case 1:
                            kunjungiWebsite_2511531011 (browser,
scanner);
                            break;
                        case 2:
```

```
                                tombolBack_2511531011 (browser);
                                break;
                        case 3:
                                halamanAktif_2511531011 (browser,
scanner);
                                break;
                        case 4:
                                System.out.println("Keluar dari
program.");
                                break;
                        default:
                                System.out.println("Pilihan tidak
valid");
                        }
                } while (choice !=4);
                scanner.close();
        }

}
```

Output:

```
=== Browser History NIM: 2511531011 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 1
Masukkan Judul: sso
Masukkan URL: sso.unand
Berhasil Mengunjungi Halaman!

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 1
Masukkan Judul: ilearn
Masukkan URL: ilearn.unand
Berhasil Mengunjungi Halaman!

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 3
Halaman aktif: ilearn ilearn.unand

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 2
Kembali dari: ilearn
```

```
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 2
Kembali dari: ilearn

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 3
Halaman aktif: sso sso.unand

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Print Menu: 4
Keluar dari program.
```

BAB III
KESIMPULAN

3.1. Kesimpulan

Penggunaan stack memiliki keuntungan sendiri, dimana kita bisa mengoperasikan data teratas tanpa mengganggu data yang paling bawah. Namun, penggunaan stack juga harus diperhatikan. Jika menggunakan stack pada suatu data yang terus berubah, maka kita akan mengalami kesulitan nyata. Namun, jika implementasi stack digunakan dengan baik, kita akan menghasilkan program yang lebih efektif dan efisien.

3.2. Saran

Karena banyaknya implementasi dari ADT, maka mahasiswa sebaiknya terus mengulang dan mempelajari setiap implementasi ADT. Mahasiswa kini hendaknya sudah mampu untuk membedakan antara array list dengan stcak. Untuk lebih mengasah keterampilan demi mendapatkan hasil program yang baik, hendaknya mahasiswa mampu dan terus belajar, salah satunya implementasi ADT.

DAFTAR PUSTAKA

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.

[2] Wahyudi, *Struktur Data dan Algoritma,* Padang: Informatika, 2026

[3] M.T. Goodrich, *Data Structures and Algorithms in Java*